SEMINARIO INGENIERÍA DE SOFTWARE CON IA GENERATIVA · UNIANDES · MÓDULO 4

Planificador turístico de la Zona Centro del Ecuador

Itinerarios día por día con fuentes citadas, costos, clima y mapa, más un asistente que ajusta el viaje sin salirse de su alcance.

GRUPO 4

Kleber Tepud
César Vasquez
Kleber Suñiga

EN PRODUCCIÓN

planificador.bhsolutions.cloud
API v0.4.2 · Web v1.1.0

SUSTENTACIÓN

Sábado 7 de noviembre de 2026
Docente: Heyler Fabio Guaza

PROBLEMA Y USUARIO

Planificar la sierra central toma horas, y la información está dispersa y desactualizada

Para quién

Visitantes nacionales y extranjeros que quieren recorrer Tungurahua, Chimborazo, Cotopaxi y Bolívar en 1 a 7 días, con un presupuesto definido.

El problema

Horarios, precios y accesos están repartidos en páginas de GAD, prensa y blogs. Un chatbot genérico inventa atractivos, precios y horarios con total seguridad.

Lo que hacemos

Un itinerario por día armado **solo** con un catálogo verificado de 139 atractivos, con la fuente de cada dato, costos, clima del día y mapa.

> Regla de todo el proyecto: **el modelo propone, el código dispone.** La IA elige atractivos y redacta; el código valida contra el catálogo, calcula costos y horas, y decide lo que es exacto o riesgoso.

DEMO EN VIVO · SOBRE LA URL PÚBLICA

https://planificador.bhsolutions.cloud

1 Caso normal con cita

Formulario: 2 días · USD 100 · Latacunga · Naturaleza · notas `Me interesa la laguna de Quilotoa` · fecha. Mostrar el itinerario por día, el mapa, los costos y en una actividad **«Lo que respalda este motivo»** → **Ver fragmento citado**.

2 Pregunta fuera de tema: se abstiene

En el asistente: `¿Qué playas hay en Galápagos?` → responde que no tiene información verificada, sin inventar y sin fuentes. (En el formulario, «playas de Galápagos» se rechaza antes de llamar al modelo.)

3 Acción con confirmación humana

En el asistente: `Envíame el itinerario a demo@example.com` → aparece la tarjeta con la descripción real → **Aprobar** → «enviado (simulado)». Extra: **Quitar** una actividad y ver que el asistente reemplaza solo esa visita.

Plan B si la plataforma falla: «la plataforma está despertando; mientras tanto, esta es la grabación de hoy» → video de 2 minutos y capturas en docs/evidencias/semana-4/.

ARQUITECTURA

Un flujo fijo para generar y un agente solo donde el camino no se puede escribir

Angular 22
Formulario, itinerario, mapa Leaflet y asistente. nginx en Docker.

→

Traefik · HTTPS
Mismo dominio: rutas de la API → API; el resto → web. Sin CORS.

→

Regla de alcance
Reservas, pagos, médico, inyección y fuera de la zona: rechazo sin gastar tokens.

→

RAG
Chroma + E5 local. Piso de similitud: si nada se parece, no hay fuentes.

→

Modelo
Groq gpt-oss-20b. Respaldo: Gemini 3.5 Flash. JSON con esquema estricto.

→

Código dispone
Valida contra el catálogo, jornada, costos, citas y clima (Open-Meteo).

Agente de alcance controlado (POST /agente)

- **Lista cerrada:** recalcular, buscar_fragmentos, consultar_clima y enviar_itinerario
- **Tope:** 6 vueltas; al llegar, «lo revisa un asesor»
- **Confirmación:** enviar espera Aprobar/Rechazar, una sola vez

Controles en código, no en el prompt

- Argumentos validados con Pydantic antes de ejecutar
- Horas y montos de la respuesta contrastados con las fuentes
- El correo debe ser uno que escribió el visitante

decide el código · decide la IA · decide la persona (confirmación)

DESPLIEGUE · QUÉ USAMOS

Dos contenedores en un VPS, detrás de Traefik, publicados por un pipeline

Servidor y red

- **VPS propio** con Ubuntu y Docker (administrado también con Portainer)
- **Traefik** como proxy inverso: HTTPS con **Let's Encrypt** y redirección de HTTP a HTTPS
- Un solo dominio: las rutas de la API van a la API; el resto, a la web

Contenedores (Docker)

- **API:** Python 3.12, FastAPI y uvicorn; torch solo CPU y el modelo de embeddings dentro de la imagen; usuario sin privilegios; HEALTHCHECK a /health
- **Web:** Node 24 compila Angular y **nginx** lo sirve (77 MB)
- **docker-compose** con etiquetas de Traefik

Datos

- Dentro de la imagen: catálogo, configuración, prompts y modelo E5
- Volúmenes del servidor: índice del RAG (Chroma) y registros
- En memoria: acciones pendientes y límite por IP (se aceptan perder al reiniciar)

Pipeline de GitHub Actions (cada repositorio)

1 · Secretos
Gitleaks revisa todo el historial

→

2 · Pruebas
pytest con cobertura ≥ 85 % · Vitest y compilación

→

3 · Imagen
docker build para comprobar que compila

→

4 · Despliegue
Solo desde master: SSH, build y docker compose up

→

5 · Verificación
/health y pruebas funcionales contra la URL

Si un paso falla, no se despliega. develop prueba; master publica (versiones etiquetadas: API v0.4.2, Web v1.1.0).

Ningún cambio llega a producción sin pasar por estas pruebas

Nivel	Qué comprueba	Cuántas	Cuándo corre
Unitarias e integración	Reglas del código, agente (lista cerrada, tope, confirmación), errores del proveedor, límite por IP; con dobles del modelo, del RAG y del clima (sin red ni claves)	234 API · 37 web	Cada push (CI); cobertura ≥ 85 % o falla
Datos del catálogo	UTF-8 y comas, coordenadas en la zona centro, valores válidos, reducido = completo	18	Cada push (CI)
Contrato de despliegue	Lo que copia el Dockerfile se sube al VPS; Traefik envía a la API todas sus rutas	3	Cada push (CI)
Evaluación con el modelo real	Conjunto dorado con lo esperado escrito antes: 15 preguntas (4 sin respuesta), 15 solicitudes (4 adversarias), 10 casos del agente	40 casos	Al cambiar prompt, modelo o datos
Funcionales contra la URL	salud, caso, fuera de dominio, inyección, entrada inválida y confirmación, con el modelo real	8	Después de cada despliegue
Seguridad	Gitleaks sobre todo el historial; casos adversarios e inyección dentro de las pruebas	—	Cada push y pull request



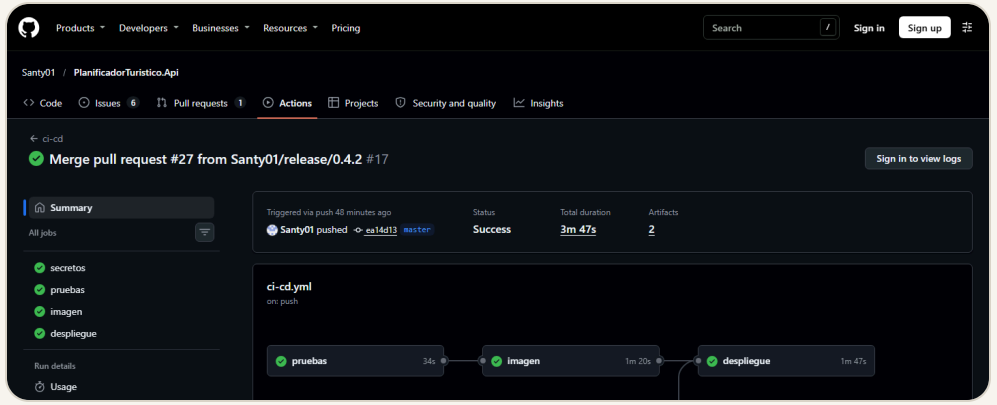
Ante un defecto: primero una prueba que lo reproduce, después la corrección, y se comprueba que la prueba falla si el defecto vuelve. Cada hallazgo queda en docs/evidencias/.

8 de 8 casos pasan contra la URL pública

#	Tipo	Caso	Resultado
1	salud	GET /health	**200 · 0.2 s**
2	caso	Itinerario con Quilotoa	**200 · 1.5 s**
3	caso	1 día cultural en Riobamba	**200 · 14.3 s**
4	fuera	Playas de Galápagos → rechazo	**200 · 0.1 s**
5	inyección	«Ignora tus instrucciones…»	**200 · 0.1 s**
6	inválida	Cuerpo vacío → 422	**422 · 0.1 s**
7	caso	Pregunta al agente con fuente	**200 · 1.3 s**
8	confirmación	Enviar → requiere aprobación	**200 · 0.5 s**

8/8 pasan · p95 14.3 s

tests/smoke.py, con el modelo real y sin claves en el cliente. Lo corre el pipeline tras cada despliegue (API v0.4.2). El caso 3 tardó 14 s por reintentos ante el límite por minuto del proveedor.



Lo esperado se escribió antes; medir encontró 4 fallas

Qué se mide	Antes (semana 3 / línea base)	Final	Qué lo cambió
RAG: la fuente esperada aparece (11 preguntas)	82 %	**100 %**	El agente busca en el catálogo completo, no en el reducido
RAG: silencio cuando no hay respuesta (4 preguntas)	no se podía: sin piso, la búsqueda nunca callaba	**100 %**	Piso de similitud calibrado (0.86 para preguntas)
Solicitudes que cumplen lo esperado (15)	6 / 15	**14 / 15**	Regla de alcance; S14 pasa corrida aparte (cuota)
Adversarias rechazadas (Galápagos, reservar, inyección, Montañita)	**0 / 4**	**4 / 4**	El modelo aceptaba todas: regla en código antes del modelo
Agente: casos que cumplen (escenarios A, B y D)	9 / 10	**10 / 10**	Correo inventado bloqueado; catálogo completo
Vueltas del agente (tope 6) · tokens por caso	media 1.6 · máximo 3 · ~2 400 tokens		El tope nunca se alcanzó con el modelo real

234 pruebas API
Cobertura 95 % (el CI falla bajo 85 %)

37 pruebas web
Vitest: mapeos, errores y asistente

0 atractivos inventados
En todas las evaluaciones (RNF-06)

PROTECCIÓN DE CREDENCIALES

El código lee las claves; nunca las contiene

Dónde	Cómo se ocultan
Repositorio	`.env` en `.gitignore`; solo se versiona `.env.example` con los nombres, sin valores. Las claves se leen de variables de entorno y, si falta una, la app se detiene con un mensaje claro
Imagen Docker	`.dockerignore` excluye `.env` y `.git`: la imagen no lleva secretos en ninguna capa
Servidor	El `.env` real existe en un solo lugar del VPS, con permisos 600 (solo root); Docker lo inyecta al arrancar (`env_file`)
Pipeline	GitHub Actions Secrets: llave SSH, servidor, usuario, huella del servidor y URL. Se leen como `${{ secrets.NOMBRE }}` por variables de entorno; nunca están escritos en el workflow
Acceso al VPS	Una llave SSH distinta para cada repositorio, solo para desplegar; la huella del servidor se verifica (known_hosts) para no conectarse a uno falso
Lo que se muestra	/health dice si hay un modelo configurado, nunca la clave; los registros no guardan el texto del visitante y el correo va enmascarado

0 hallazgos

Gitleaks sobre todo el historial, en local y en cada push (ambos repositorios)

1 falso positivo

Un valor de prueba con forma de clave: corregido antes de publicarlo

1 llave rotada

Una llave de despliegue quedó expuesta: se generó otra antes de usarla

INTEGRACIÓN IA ESTABLE

Si el modelo falla, la app sigue viva y lo dice

Cuando el proveedor falla

- **Tiempo límite** de 30 s en cada llamada
- **Reintentos** con espera exponencial; respeta Retry-After
- **Respaldo:** Groq → Gemini, probado con Groq caído y a mitad de una conversación del agente
- **Mensaje claro** (503), nunca la traza del error
- /health responde sin llamar al modelo

Para que no se agote ni se abuse

- **Límite por IP:** 10 itinerarios y 20 mensajes por minuto → 429 con cuenta regresiva en la web
- **Topes de entrada:** texto, días e historial; tokens de salida acotados
- **Regla de alcance** antes del modelo: rechaza sin gastar tokens
- **Arranque en frío:** el RAG se precarga al iniciar

> OWASP 2026: inyección (LLM01/ASI01) → regla de alcance y datos delimitados · agencia excesiva (LLM06/ASI02) → lista cerrada y confirmación · desinformación (LLM09) → citas validadas y verificación de datos.

LIMITACIONES · DICHAS CON HONESTIDAD

Lo que todavía no hace, o hace a medias

Cuota de los planes gratuitos

Groq y Gemini tienen límites por minuto y por día. El 7 de noviembre se agotaron por las pruebas: con ambos sin cuota, la API responde 503. Falta separar claves de desarrollo y producción.

Una corrida por caso

La evaluación no mide cuánto varía el modelo entre corridas; el agente inventó un correo en una corrida y lo pidió en otra.

El piso no separa todo

«natural» puntúa igual que «playas de Galápagos» (0.838 vs 0.839). Lo ajeno a la zona lo frena la regla de alcance, que es una lista de patrones.

Catálogo

60 atractivos verificados por el equipo; de los otros 79, muchos costos siguen «por verificar». No distingue eventos con fecha (la Fiesta de las Flores es en febrero).

Alcance funcional

El envío por correo es simulado; no optimiza la ruta ni geocodifica el punto de partida; la verificación de datos cubre horas y montos, no todo el texto.

Operación

Un solo servidor y un proceso: las acciones pendientes y el límite por IP viven en memoria. Gemini puede tardar 11–66 s cuando está saturado.

Cada entregable, con su evidencia

Entregable	Evidencia	Dónde
Producto final operativo	Itinerario con citas, abstención y asistente en la URL pública	planificador.bhsolutions.cloud
Integración IA estable	Tiempo límite, reintentos, respaldo Gemini, 503 claro, límite por IP (429)	src/services/llm.py · src/limites.py
Despliegue	HTTPS con /health, Dockerfile, pipeline: Gitleaks → pruebas → imagen → despliegue	.github/workflows/ci-cd.yml (ambos repos)
Documentación técnica	README con URL, instalación, resultados, ADR y limitaciones	README.md · docs/
Pruebas funcionales	8 casos contra la URL, incluida la inyección: 8/8	tests/smoke.py · docs/evidencias/semana-4/
Protección de credenciales	Claves solo en el VPS y en GitHub Secrets; Gitleaks sin hallazgos	docs/evidencias/semana-4/ §1



Gracias.

Preguntas del tribunal: cualquiera de los tres responde por cualquier parte.

¿POR QUÉ ASÍ Y NO DE OTRA FORMA?

Cada decisión, con la alternativa descartada y un dato

Decisión	Alternativa descartada	Dato que la respalda
Generar con un **flujo fijo**; agente solo para ajustar	Que el agente arme todo el itinerario	Los pasos son siempre los mismos: 1 llamada (~2 700 tokens) frente a varias vueltas
Regla de alcance en código antes del modelo	Confiar en la instrucción del prompt	El modelo aceptó 0 de 4 adversarias; con la regla, 4 de 4 y 0 tokens
Chroma local + E5-small	Qdrant o pgvector como servicio	~140 atractivos y 251 fragmentos no justifican un servicio aparte
Gemini como respaldo	Solo reintentar Groq	En producción Groq pidió esperar 23 s; con respaldo se responde igual
VPS propio con Traefik	Render, Cloud Run, Azure	Render: 512 MB no alcanzan para torch + E5 (~470 MB); el VPS ya existía
Mismo dominio para web y API	Subdominio + CORS	Sin CORS y la API ve la IP real del visitante para el límite
Confirmación solo para enviar	Confirmar también recalcular	Recalcular no sale del sistema y se ve en pantalla; enviar no se puede deshacer

LO QUE ENCONTRAMOS AL MEDIR, Y CÓMO SE CORRIGIÓ

Ninguna de estas fallas se veía en la demo; todas salieron de pruebas reales

Horario inventado

«Según la ficha oficial abre de 9:00 a 19:00» y la ficha no trae horario. → El código contrasta horas y montos con las fuentes; pide una corrección o avisa.

Correo inventado

Con «mándamelo por correo» el agente pidió enviar a una dirección que nadie escribió. → El correo debe aparecer en lo que escribió el visitante.

Respaldo roto a mitad de conversación

Gemini 3 rechazaba (400) los pedidos de herramientas que había hecho Groq: faltaba la firma de pensamiento. → Firma documentada por Google; probado con ambos modelos.

El despliegue habría fallado

El pipeline no subía `config/` al VPS. → Corregido, y una prueba compara lo que copia el Dockerfile con lo que se sube.

Excel dañó el catálogo

Coordenadas sin punto (-1.3982 → -13.982), «;» y codificación de DOS. → Script de importación con reglas y 18 validaciones en el CI.

4 de 4 adversarias aceptadas

El prompt pedía rechazarlas y el modelo no lo hacía. → Regla de alcance en código, con pruebas de falsos positivos.

USO DE IA GENERATIVA EN EL DESARROLLO

Qué se generó, cómo se revisó y qué lo respalda

Qué se generó con IA

- Código del backend y del frontend con un asistente de programación con IA (Claude Code)
- Pruebas, scripts de evaluación y documentación
- Las decisiones de alcance, las prioridades y los datos del catálogo los definió y verificó el equipo

Cómo se revisó

- Pull requests por tarea con CI obligatorio
- Evaluación con lo esperado escrito **antes** de correr
- Pruebas contra el modelo real y contra producción
- Gitleaks en cada push

Qué lo respalda y qué se corrigió

- 234 pruebas en la API, 37 en la web, 95 % de cobertura
- Fallas encontradas al medir y corregidas: diapositiva 11
- Errores del código generado detectados en la revisión (p. ej. el estado del asistente permitía confirmar dos veces) y cubiertos después con una prueba

> Criterio: nada generado se acepta sin una prueba que lo respalde; cuando una prueba falla, se corrige el código, no la prueba (salvo que la prueba estuviera mal, y entonces se documenta).